JavaScript Iterators

❮ Previous Next ❯

The Iterator Object

An **Iterator** is an object that provides a standard way to access elements **sequentially**.

An Iterator must adheres to the **Iterator Protocol**: It must have a **next()** method.

The next() Method

The **next()** method returns an object with two properties:

- The **value** property holds the **next value** in the iteration sequence.
- The **done** property returns `false` if there are more elements to iterate over, otherwise it returns `true`.

The For Of Loop

The JavaScript `for..of` statement loops through the elements of an iterable object.

Syntax

```
for (variable of iterable) {
  // code block to be executed
}
```

Technically, iterables must implement the `Symbol.iterator` method.

In JavaScript the following are iterables:

- Strings
- Arrays
- Typed Arrays
- Sets
- Maps

Because their prototype objects have a `Symbol.iterator` method:

Iterators provide a controlled way to work with data sequences, enabling custom iteration logic for various data structures.

They are particularly useful for handling streams of data, lazy computation of values, and building custom data structures with defined iteration behaviors.

Helper Functions

JavaScript 2025 (ECMAScript 2025) officially approved a set of new Iterator Helper methods that significantly enhance the functionality of iterators in JavaScript.

The methods provide a more functional and efficient way to work with iterable objects, including generators, by allowing direct manipulation and transformation without first converting them to arrays:

Function	Description
drop()	Returns an iterator that skips a specified number of elements before yielding the rest
every()	Returns `true` if all elements satisfy a test function
filter()	Returns an iterator containing elements that satisfy a filter function
find()	Returns the first element that satisfies a test function
flatMap()	Returns an iterator by mapping each element and then flattening the results
forEach()	Executes a function once for each element in the iterator.
from()	creates an iterator object from an iterable
map()	Returns an iterator with all elements transformed by a map function
reduce()	Applies a reducer function against each element to reduce it to a single value

The Iterator.from() Method

The `Iterator.from()` creates an iterator object from an existing iterable or iterator object.

Example

```
// Create an iterator
const myIterator = Iterator.from([1, 2, 3]);

// Iterate over the elements
let text = "";
for (const x of myIterator) {
  text += x;
}
```

Try it Yourself »

The filter() Method

The `filter()` method returns a new iterator containing elements that satisfy a filter function.

Example

```
// Create an iterator
const myIterator = Iterator.from([32, 33, 16, 40]);

// Filter the iterator
const filteredIterator = myIterator.filter(x => x > 18);
```

Try it Yourself »

The map() Method

The `map()` method returns a new iterator with all elements transformed by a map function.

```
// Create an iterator
const myIterator = Iterator.from("123456789");

// Now you can use the map method
const mappedIterator = myIterator.map(x => x * 2);
```

Try it Yourself »

The flatMap() Method

The `flatMap()` method returns a new iterator by mapping each element and then flattening the results into a single iterator.

Example

```
// Create an iterator
const myIterator = Iterator.from([1, 2, 3, 4, 5, 6]);

// Map the Iterator
const mappedIterator = myIterator.flatMap(x => [x, x * 10]);
```

Try it Yourself »

The take() Method

The `take()` method returns a new iterator that yields at most a specified number of elements.

Example

```
const myIterator = Iterator.from([1, 2, 3, 4, 5, 6]);

// Take the first five elements
const firstFive = myIterator.take(5);
```

Try it Yourself »

The `drop()` method returns a new iterator that skips a specified number of elements before yielding the rest.

Example

```
// Create an iterator
const myIterator = Iterator.from([1, 2, 3, 4, 5, 6]);

// Remove the first five
const firstFive = myIterator.drop(5);
```

Try it Yourself »

The find() Method

The `find(fn)` method returns the first element that satisfies a test function.

Example

```
// Create an iterator
const myIterator = Iterator.from([3, 10, 18, 30, 20]);

// Find first greater than 18
let result = myIterator.find(x => x > 18);
```

The reduce() Method

The `reduce()` method applies a reducer function against an accumulator and each element to reduce it to a single value.

Example

```
// Create an Iterator
const myIterator = Iterator.from([175, 50, 25]);

// Reduce the Iterator
let result = myIterator.reduce(myFunc);
```

Try it Yourself »

The every() Method

The `every(fn)` method returns `true` if all elements in the iterator satisfy the provided test function.

Example

```
// Create an Iterator
const myIterator = Iterator.from("123456789");

// Is every Element greater than 7?
let result = myIterator.every(x => x > 7);
```

Try it Yourself »

The some() Method

The `some()` method returns `true` if at least one element in the iterator satisfies the provided test function.

```
// Create an iterator
const myIterator = Iterator.from("123456789");

// Is some Element greater than 7?
let result = myIterator.some(x => x > 7);
```

Try it Yourself »

The forEach() Method

The `forEach()` method executes a provided function once for each element in the iterator.

Example

```
// Create an iterator
const myIterator = Iterator.from("123456789");

// Iterate over all elements
let text = "";
myIterator.forEach (x => text += x);
```

Try it Yourself »

Note

Iterators bring the **iteration concept** directly into the core JavaScript language and provide a mechanism for customizing the behavior of for...of.

COLOR PICKER

PLUS SPACES GET CERTIFIED

FOR TEACHERS FOR BUSINESS

CONTACT US

Top Tutorials

HTML Tutorial
CSS Tutorial
JavaScript Tutorial
How To Tutorial
SQL Tutorial
Python Tutorial
W3.CSS Tutorial
Bootstrap Tutorial
PHP Tutorial
Java Tutorial
C++ Tutorial
jQuery Tutorial

Top References

HTML Reference
CSS Reference
JavaScript Reference
SQL Reference
Python Reference
W3.CSS Reference
Bootstrap Reference
PHP Reference
HTML Colors
Java Reference
Angular Reference
jQuery Reference

Top Examples

HTML Examples
CSS Examples
JavaScript Examples
How To Examples
SQL Examples
Python Examples
W3.CSS Examples
Bootstrap Examples
PHP Examples
Java Examples
XML Examples
jQuery Examples

Get Certified

HTML Certificate
CSS Certificate
JavaScript Certificate
Front End Certificate
SQL Certificate
Python Certificate
PHP Certificate
jQuery Certificate
Java Certificate
C++ Certificate
C# Certificate
XML Certificate

Copyright 1999-2025 by Refsnes Data. All Rights Reserved. W3Schools is Powered by W3.CSS.